Gyroresonance emission, step by step

with a simple Python code that anyone can read

Soumitra Hazra

Companion to radiosynth 2.2.0 — *From MHD to MHz* —
https://github.com/soumitrahazra/radiosynth

This note explains how to compute the thermal gyroresonance radio emission of a sunspot, one step at a time, and gives a short Python program, `gyroresonance_simple.py`, that does exactly these steps and nothing else. The program needs only `numpy` and `matplotlib`; it is about 300 lines long, most of them comments, and it runs in about half a minute:

```
python gyroresonance_simple.py # writes gyroresonance_simple_fig.png
```

Each section below is one `STEP` of the program: first the idea, then the equation, then the lines of code that compute it. The full program is printed in the Appendix. It uses the same physics as radiosynth and gives the same images to 0.01 % (see *Check against radiosynth*), but it is written for reading, not for speed or generality: the model is a flat box seen straight from above.

The plan

Seven steps from a magnetic field to a radio image

0. Constants.

1. A model of an active region: density, temperature and magnetic field in a box.

2. One straight line of sight per pixel.

3. The plasma along each line: field strength, angle to the line of sight, gyrofrequency.

4. The ordinary (free–free) absorption of the two wave modes.

5. The gyroresonance layers crossed by each line, and how opaque they are.

6. The radiative transfer: add up the light along each line, for right- and left-handed waves.

7. What the telescope sees: brightness temperature, circular polarisation and flux.

STEP 0. Constants

Everything in cgs units

We use cgs units, like most solar radio papers. Two combinations appear again and again:

$$\nu_B = \frac{eB}{2\pi m_e c} = 2.80 \times 10^6 \text{ Hz} \times \frac{B}{1\,\text{G}}, \qquad \frac{\pi e^2}{m_e c} = 0.02654 \text{ cm}^2\,\text{s}^{-1}. \tag{1}$$

The first is the *gyrofrequency*: how many times per second an electron goes around a field line. The second sets the strength of the absorption.

```
NU_B_PER_GAUSS = E / (2.0 * math.pi * ME * C) # 2.80e6 Hz per gauss
GR_PREFACTOR = math.pi * E**2 / (ME * C) # 0.02654 cm^2/s
```

STEP 1. The model

A box above two sunspots

The box is 140 Mm wide and 150 Mm high; x, y are horizontal and z is the height above the photosphere, all in Mm.

Density falls off exponentially with height, and the **temperature** is the same everywhere (to keep things simple):

$$n_e(z) = n_0\, e^{-z/H}, \quad n_0 = 2 \times 10^9 \text{ cm}^{-3}, \; H = 35 \text{ Mm}; \qquad T_e = 2.5 \text{ MK}. \tag{2}$$

The magnetic field is that of a small bar magnet (a dipole $\boldsymbol{m}$, pointing along x) buried 35 Mm below the surface. Its two ends make two spots of opposite polarity:

$$\boldsymbol{B}(\boldsymbol{r}) = \frac{3(\boldsymbol{m}\cdot\boldsymbol{r})\,\boldsymbol{r} - \boldsymbol{m}\,r^2}{r^5}, \tag{3}$$

with $\boldsymbol{r}$ measured from the dipole, scaled so that $|\boldsymbol{B}| = 2500$ G at the surface right above it. Fig. 1 shows the field: strongest near the surface, weakening with height.

```python
def density(z):
    return N0 * np.exp(-z / H_N)

def magnetic_field(x, y, z):
    rx, ry, rz = x, y, z + DEPTH # position relative to the dipole
    r2 = rx**2 + ry**2 + rz**2
    r5 = r2**2.5
    mdotr = mx * rx
    bx = (3.0 * mdotr * rx - mx * r2) / r5
    by = (3.0 * mdotr * ry) / r5
    bz = (3.0 * mdotr * rz) / r5
    ... # scaled to B_MAX
```

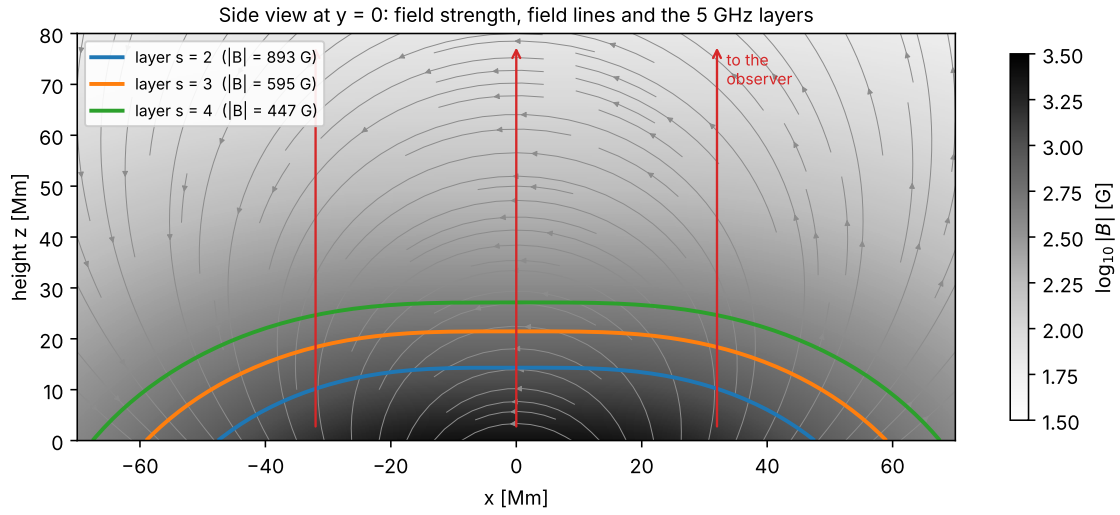


Figure 1: Side view of the model ($y = 0$): field strength (grey), field lines, and the three surfaces where gyroresonance happens at 5 GHz (Step 5). The red arrows are three lines of sight, looking straight down at the spots ($x = \pm 32$ Mm) and between them ($x = 0$).

Remember: any MHD model can replace these three functions; radiosynth does exactly that with the density, temperature and field of BATS-R-US, PLUTO or other simulations.

STEP 2. The lines of sight

One straight line per pixel, looking down

We look straight down from above: one vertical line per pixel, 100×100 pixels. The wave travels *up*, towards us, so the direction towards the observer is

$$\hat{\boldsymbol{k}} = (0, 0, 1). \tag{4}$$

At microwave frequencies the plasma is far too thin to bend the rays, so straight lines are fine. Each line is sampled at 601 heights from the top ($z = 150\,\mathrm{Mm}$, point 0, nearest to us) down to the photosphere, every 0.25 Mm.

```
c = -0.5 * FOV + (np.arange(NPIX) + 0.5) * FOV / NPIX # pixel centres
X, Y = np.meshgrid(c, c)
z = np.linspace(Z_TOP, 0.0, NZ + 1) # top -> photosphere
```

STEP 3. The plasma along each line

What every sample point needs

At each point we need the field strength $B = |\boldsymbol{B}|$, the angle θ between the field and the line of sight, and three frequencies compared with the observing frequency ν:

$$\cos\theta = \frac{\boldsymbol{B}\cdot\hat{\boldsymbol{k}}}{B} = \frac{B_z}{B}, \qquad \nu_B = 2.80\,\mathrm{MHz} \times B, \qquad X = \frac{\nu_p^2}{\nu^2}, \quad \nu_p^2 = \frac{e^2 n_e}{\pi m_e}, \qquad Y = \frac{\nu_B}{\nu}. \tag{5}$$

ν_p is the plasma frequency. X tells how dense the plasma is for this wave (it can only travel where X is small enough), and Y how strong the field is ($Y = 1/s$ exactly on the s-th layer).

```
cos_theta = bz / b # angle between B and k = (0, 0, 1)
nu_b = NU_B_PER_GAUSS * b # gyrofrequency [Hz]
nu_p2 = E**2 * ne / (math.pi * ME) # plasma frequency squared
Xmi = nu_p2 / freq**2 # X
Ymi = nu_b / freq # Y
```

STEP 4. Free–free absorption of the two modes

The "background" opacity

In a magnetic field a radio wave can only travel in two ways, the two *modes*: the **x-mode** (extraordinary), whose electric field rotates with the gyrating electrons, and the **o-mode** (ordinary). Even without gyroresonance, the ordinary collisions of electrons with ions absorb the waves (free–free absorption, Dulk 1985):

$$\kappa_{\mathrm{ff}} = 9.78 \times 10^{-3}\, \frac{n_e^2}{\nu^2 T_e^{3/2}}\, (24.5 + \ln T_e - \ln \nu) \ \ \mathrm{cm}^{-1}, \tag{6}$$

slightly more for the x-mode and less for the o-mode:

$$\kappa_x = \frac{\kappa_{\mathrm{ff}}}{(1-q)^2}, \qquad \kappa_o = \frac{\kappa_{\mathrm{ff}}}{(1+q)^2}, \qquad q = Y|\cos\theta| \ (\le 0.9). \tag{7}$$

For each step between two sample points (length $\Delta\ell$) the optical depth is $\Delta\tau_{\mathrm{ff}} = \kappa\,\Delta\ell$, with κ taken at the middle of the step.

```
kappa = 9.78e-3 * ne**2 / (freq**2 * te**1.5) * G
q = np.minimum(Ymi * np.abs(cos_theta), 0.9)
kappa_x, kappa_o = kappa / (1 - q)**2, kappa / (1 + q)**2
dtau_x, dtau_o = kappa_x * dl, kappa_o * dl
```

For our model this alone gives at most 0.23 MK at 5 GHz — far too faint to explain a sunspot. The next step adds what is missing.

STEP 5. The gyroresonance layers

The heart of the calculation

(a) Where. Electrons spiralling in the field absorb and emit at their gyrofrequency and its multiples. A wave of frequency ν is affected only where

$$\nu = s\,\nu_B, \qquad s = 1, 2, 3, \ldots \qquad \Longleftrightarrow \qquad B = B_s = \frac{\nu}{s \times 2.80\,\mathrm{MHz/G}}. \tag{8}$$

These are thin surfaces of constant field strength, the *layers*. At 5 GHz they are at $B_2 = 893$ G, $B_3 = 595$ G, $B_4 = 447$ G (coloured lines in Fig. 1).

(b) Finding them along a line. Between two neighbouring points k and $k+1$ there is a layer of harmonic s if $s\nu_B - \nu$ changes sign:

$$(s\,\nu_{B,k} - \nu)(s\,\nu_{B,k+1} - \nu) < 0. \tag{9}$$

Where exactly? By linear interpolation, at the fraction $u = (\nu/s - \nu_{B,k})/(\nu_{B,k+1} - \nu_{B,k})$ of the step, and we take n_e, T_e, $\cos\theta$ and X there.

(c) How thick. The layer is thin, and its strength depends on how fast B changes along the line, measured by the *magnetic scale length*

$$L_B = \left|\frac{B}{\mathrm{d}B/\mathrm{d}\ell}\right| \;\approx\; \frac{B_s\,\Delta\ell}{|B_{k+1} - B_k|}. \tag{10}$$

(d) How opaque. The optical depth of the layer is

$$\boxed{\tau_s = \frac{\pi e^2}{m_e c}\,\frac{n_e\,L_B}{\nu}\,\frac{s^2}{s!}\left(\frac{s^2\beta_T^2\sin^2\theta}{2}\right)^{s-1} F(\theta), \qquad \beta_T^2 = \frac{k_\mathrm{B}T_e}{m_e c^2}} \tag{11}$$

($\beta_T^2 \approx 4\times10^{-4}$ at 2.5 MK). The bracket raised to the power $s-1$ makes each higher harmonic several hundred times weaker, so only $s = 2$, 3 and sometimes 4 matter. The factor $\sin^2\theta$ makes a layer seen *along* the field almost transparent.

(e) Which mode. F says how well each mode couples to the electrons:

$$F_x = \frac{(1 + |T||\cos\theta|)^2}{1 + T^2}, \qquad F_o = \frac{(|T| - |\cos\theta|)^2}{1 + T^2}, \tag{12}$$

where $|T|$ measures how circular the x-mode is (1: circular, 0: linear),

$$|T| = \frac{2(1-X)|\cos\theta|}{Y\sin^2\theta + \sqrt{Y^2\sin^4\theta + 4(1-X)^2\cos^2\theta}}, \tag{13}$$

with $Y = 1/s$. Along the field $F_x = 2$ and $F_o = 0$: the x-mode, rotating with the electrons, is absorbed far more. (On the $s = 1$ layer the x-mode cannot exist at all, so its τ is set to 0.)

```
crosses = (s * f0 - freq) * (s * f1 - freq) < 0 # (b) a layer in this step?
u = (freq / s - f0) / (f1 - f0) # where in the step
ne_c, te_c, cos_c, X_c = at(ne), at(te), at(cos_theta), at(Xmi)
b_s = freq / (s * NU_B_PER_GAUSS) # (a) |B| on the layer
l_b = b_s * dl / np.abs(b[..., 1:] - b[..., :-1]) # (c) L_B
beta2 = KB * te / (ME * C**2) # (d) tau_s
tau = (GR_PREFACTOR * ne * l_b / freq * s**2 / math.factorial(s)
      * (0.5 * s**2 * beta2 * sin2)**(s - 1) * F) # F from (e)
```

The layer optical depths of a step are added to its free–free optical depth, for each mode.

Remember: at each frequency, the emission comes from a few thin layers of constant $|B|$; the x-mode sees them as opaque walls, the o-mode as much thinner veils.

STEP 6. Adding up the light

Radiative transfer, from the observer inwards

How bright each step glows. The electrons are thermal, so each mode glows exactly as much as it absorbs (Kirchhoff's law), at the temperature of the gas. It is easiest to count everything in temperature units: for one polarisation, the "brightness temperature" $T_p = I_p c^2/(k_\mathrm{B}\nu^2)$. A step whose optical depth comes from several parts (free–free and maybe a layer) glows at their optical-depth-weighted temperature

$$T_\mathrm{step} = \frac{\sum_i \tau_i T_i}{\sum_i \tau_i} \tag{14}$$

(just T_e in our isothermal model).

Walking along the line. Starting at the observer, each step adds its glow, dimmed by everything in front of it:

$$\boxed{T_p \;\leftarrow\; T_p + T_\mathrm{step}\left(1 - e^{-\Delta\tau}\right) e^{-\tau}, \qquad \tau \;\leftarrow\; \tau + \Delta\tau} \tag{15}$$

A transparent step ($\Delta\tau \ll 1$) adds little; an opaque one adds its full temperature, but only if the gas in front (τ) has not already hidden it. Behind an opaque layer, nothing more is seen.

Right- and left-handed light. A telescope measures right-hand (R) and left-hand (L) circular polarisation. Where the field points towards us ($\cos\theta > 0$), the x-mode is right-handed; where it points away, left-handed. So in each step R uses the x-mode optical depth if $\cos\theta > 0$ and the o-mode one otherwise, and L the opposite. Finally, a wave stops where its mode cannot exist (o-mode: $X \geq 1$; x-mode: $X \geq 1 - Y$).

```
pos = mid(cos_theta) > 0 # field towards us?
dtau_R = np.where(pos, dtau_x, dtau_o) # R: x-mode there, else o-mode
dtau_L = np.where(pos, dtau_o, dtau_x)
for k in range(nk): # walk along the line
    d = dtau[..., k]
    t_step = tT[..., k] / d
    t_pol += t_step * (1 - np.exp(-d)) * np.exp(-tau)
    tau += d
```

STEP 7. What the telescope sees

Brightness, polarisation, flux

Each polarisation carries half of the light, so the brightness temperature of the total light (Stokes I) and of the circular polarisation (Stokes V) are

$$T_b = \tfrac{1}{2}\,(T_R + T_L), \qquad T_V = \tfrac{1}{2}\,(T_R - T_L), \qquad \frac{V}{I} = \frac{T_R - T_L}{T_R + T_L}. \tag{16}$$

The flux density at the Earth, adding all pixels of solid angle $\Delta\Omega$ (= (pixel size/1 AU)^2), is

$$F = \frac{2k_\mathrm{B}\nu^2}{c^2} \sum_\mathrm{pixels} T_b\,\Delta\Omega, \qquad 1\ \mathrm{sfu} = 10^{-19}\ \mathrm{erg\,s^{-1}cm^{-2}Hz^{-1}}. \tag{17}$$

```
tb = 0.5 * (t_R + t_L)
tb_v = 0.5 * (t_R - t_L)
pol = tb_v / tb
flux = 2 * KB * freq**2 / C**2 * tb.sum() * d_omega / 1e-19 # sfu
```

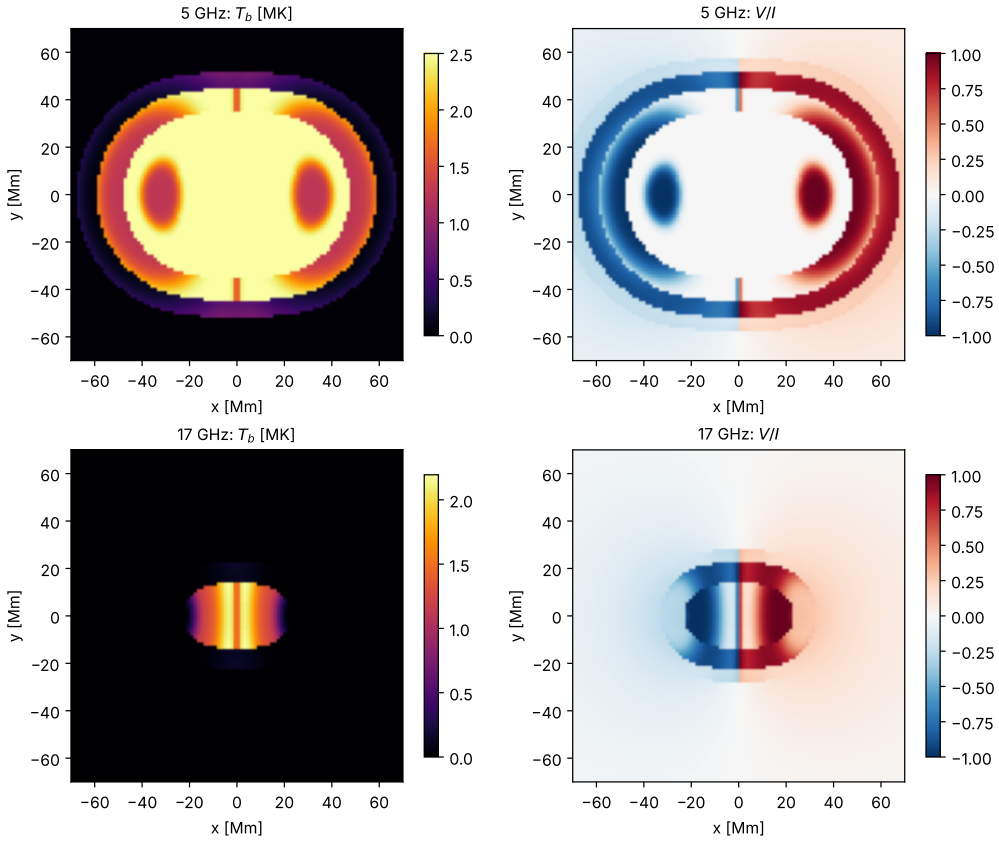


Figure 2: The output of `gyroresonance_simple.py`: brightness temperature and circular polarisation V/I at 5 GHz (top) and 17 GHz (bottom), seen from above.

Reading the result (Fig. 2):

- **At 5 GHz most of the region is at $T_b = 2.5$ MK $= T_e$**: every line crosses the $s = 2$ or $s = 3$ layer, which is opaque in both modes (for example $\tau_R \approx 1.7 \times 10^4$, $\tau_L \approx 130$ at $x = y = 15$ Mm), so we see the temperature of the corona and no polarisation. The total flux is 68 sfu, against 2.6 sfu from free–free alone.
- **The ring at the edge** is the $s = 3$ and $s = 4$ layers seen where they are only partly opaque, and mostly in the x-mode, hence strongly polarised.
- **The spot centres ($x = \pm 32$ Mm) are darker and almost 100 % polarised.** There the field points straight up or down, along the line of sight: $\sin\theta \to 0$, the layers become thin, and only the x-mode is still opaque ($\tau_R = 29$, $\tau_L = 0.006$). So $T_R = 2.5$ MK, $T_L \approx 0$, $T_b = 1.26$ MK and $V/I = +0.99$ over the spot whose field points towards us, -0.99 over the other. The total V of the symmetric pair is zero.
- **At 17 GHz the source shrinks** to the core where $B > B_3 = 2024$ G, close to the surface between the spots; the peak is 2.2 MK and the flux 62 sfu.

Check against radiosynth

Same answer, to 0.01 %

The script `check_with_radiosynth.py` (next to the program) puts the same model on a radiosynth grid and computes the image with `rs.make_gr_image`, looking from above with parallel rays. A low-resolution version runs with the tests of the package (`tests/test_simple_gyroresonance.py`), so the two codes cannot drift apart unnoticed. At full resolution:

	5 GHz		17 GHz	
	simple code	radiosynth	simple code	radiosynth
max T_b [MK]	2.500	2.500	2.195	2.200
flux [sfu]	67.88	67.88	62.19	62.20
sum of $\lvert V\rvert$ (ratio)	0.9999		1.0012	

No pixel differs by more than 5 % of the peak. The tiny differences come from radiosynth interpolating the model on a grid, while the simple code uses the exact formulas.

Try it yourself

Things to change in the program

- `TE`: a hotter corona makes brighter layers ($T_b = T_e$) and more opaque high harmonics (β_T^2 grows).
- `B_MAX`: a stronger field moves the layers higher and lets them exist at higher frequencies.
- `freqs`: scan from 2 to 20 GHz and watch the source shrink and the layers sink.
- `s_max` in `gyroresonance_image`: set it to 0 to keep only free–free, or to 2 to see what the $s = 3$ layer adds.
- `temperature(z)`: add a cool transition region below a few Mm and see the spot centres become darker still.

What the simple code leaves out. It looks straight down at a flat box with an analytic model. radiosynth does the same calculation for any observer position, for real MHD output on Cartesian or spherical grids, with the choice of strong mode coupling, more harmonics, a body temperature behind the lines of sight, and diagnostics such as the dominant harmonic in every pixel. The equations are the same.

Further reading. White & Kundu 1997, Sol. Phys. 174, 31; Dulk 1985, ARA&A 23, 169; Zheleznyakov 1970, *Radio Emission of the Sun and Planets*; and the companion notes *Thermal gyroresonance emission in radiosynth* (every equation in detail) and *The story of a sunspot at microwaves*.

Appendix: the full program

```
# radiosynth - From MHD to MHz: a simple, self-contained gyroresonance calculation
# Copyright (C) 2026 Soumitra Hazra
# SPDX-License-Identifier: GPL-3.0-or-later
"""
gyroresonance_simple.py -- thermal gyroresonance + free-free emission of a sunspot,
                           computed step by step, for teaching.

Needs only numpy and matplotlib. Run:

    python gyroresonance_simple.py (writes gyroresonance_simple_fig.png and .pdf)

What it does (each STEP below matches a section of gyroresonance_simple.pdf):

  STEP 0 physical constants (cgs units, as in most solar radio papers)
  STEP 1 a simple model of an active region: a box above the photosphere with
         an exponential atmosphere and the field of a buried dipole (two spots)
  STEP 2 one straight, vertical line of sight per pixel, seen from above
  STEP 3 along each line: n_e, T_e, |B|, the angle theta, nu_B, X and Y
  STEP 4 free-free absorption of the two modes (x and o) in every step
  STEP 5 the gyroresonance layers nu = s nu_B crossed in every step, and their
         optical depths
  STEP 6 radiative transfer for right- and left-hand circular polarisation
  STEP 7 brightness temperature, Stokes V, polarisation, flux; figures

This is the same physics as radiosynth.make_gr_image, written as plainly as
possible (and checked against it): a straight, vertical view of a flat box.

TEACHING VERSION. It is meant for reading and for experiments. For images of
real MHD models, any observer position, strong mode coupling and the other
options, use radiosynth.make_gr_image.
"""
import math

import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# ============================================================================
# STEP 0. Constants (cgs)
# ============================================================================
E = 4.80320e-10 # electron charge [esu]
ME = 9.10938e-28 # electron mass [g]
C = 2.99792458e10 # speed of light [cm/s]
KB = 1.380649e-16 # Boltzmann constant [erg/K]
MM = 1.0e8 # 1 Mm in cm

NU_B_PER_GAUSS = E / (2.0 * math.pi * ME * C) # gyrofrequency per gauss = 2.80e6 Hz/G
GR_PREFACTOR = math.pi * E**2 / (ME * C) # pi e^2/(m_e c) = 0.02654 cm^2/s

# ============================================================================
# STEP 1. The model: a box above a sunspot pair
# ============================================================================
# x, y : horizontal [Mm], z : height above the photosphere [Mm]
N0 = 2.0e9 # electron density at the base [cm^-3]
H_N = 35.0 # density scale height [Mm]
TE = 2.5e6 # coronal temperature [K] (isothermal, to keep it simple)
B_MAX = 2500.0 # field strength at the surface right above the dipole [G]
DEPTH = 35.0 # depth of the buried dipole [Mm]
```

```
def density(z):
    """Electron density [cm^-3] at height z [Mm]: n_e = N0 exp(-z/H_N)."""
    return N0 * np.exp(-z / H_N)


def temperature(z):
    """Electron temperature [K]: the same everywhere."""
    return np.full_like(z, TE)


def magnetic_field(x, y, z):
    """Field [G] of a horizontal dipole buried at depth DEPTH, pointing along x.

    A dipole m at the origin gives B = (3 (m.r) r - m r^2) / r^5. Its two
    poles look like two spots of opposite polarity at x < 0 and x > 0.
    It is scaled so that |B| = B_MAX at the surface right above the dipole."""
    rx, ry, rz = x, y, z + DEPTH # position relative to the dipole
    r2 = rx**2 + ry**2 + rz**2
    r5 = r2**2.5
    mx = 1.0 # dipole moment along x (arbitrary units)
    mdotr = mx * rx
    bx = (3.0 * mdotr * rx - mx * r2) / r5
    by = (3.0 * mdotr * ry) / r5
    bz = (3.0 * mdotr * rz) / r5
    scale = B_MAX * DEPTH**3 # above the dipole: |B| = 1/DEPTH^3
    return scale * bx, scale * by, scale * bz


# ============================================================================
# STEP 2. The lines of sight
# ============================================================================
# We look straight down from above: one vertical line per pixel. The wave
# travels upwards, towards us, so the unit vector towards the observer is
# k = (0, 0, 1). We sample each line at NZ+1 heights from the top (node 0,
# nearest to the observer) to the photosphere.
FOV = 140.0 # field of view [Mm] (square)
NPIX = 100 # pixels along each side
Z_TOP = 150.0 # top of the box [Mm]
NZ = 600 # number of steps along each line (step = 0.25 Mm)


def lines_of_sight():
    """Pixel centres X, Y [Mm] and the heights z of the sample points [Mm]."""
    c = -0.5 * FOV + (np.arange(NPIX) + 0.5) * FOV / NPIX
    X, Y = np.meshgrid(c, c) # (NPIX, NPIX)
    z = np.linspace(Z_TOP, 0.0, NZ + 1) # from the top down to the photosphere
    return c, X, Y, z


# ============================================================================
# STEP 3. The plasma along every line
# ============================================================================
def sample(X, Y, z, freq):
    """n_e, T_e, |B|, cos(theta), nu_B, X = (nu_p/nu)^2, Y = nu_B/nu at every
    sample point; arrays of shape (NPIX, NPIX, NZ+1)."""
    xx = X[:, :, None] + 0.0 * z
    yy = Y[:, :, None] + 0.0 * z
    zz = np.broadcast_to(z, xx.shape)
    ne = density(zz)
    te = temperature(zz)
    bx, by, bz = magnetic_field(xx, yy, zz)
    b = np.sqrt(bx**2 + by**2 + bz**2)
    cos_theta = bz / np.maximum(b, 1e-30) # angle between B and k = (0, 0, 1)
```

```
    nu_b = NU_B_PER_GAUSS * b # gyrofrequency [Hz]
    nu_p2 = E**2 * ne / (math.pi * ME) # plasma frequency squared [Hz^2]
    Xmi = nu_p2 / freq**2 # X = (nu_p/nu)^2
    Ymi = nu_b / freq # Y = nu_B/nu
    return ne, te, b, cos_theta, nu_b, Xmi, Ymi


# ============================================================================
# STEP 4. Free-free absorption of the two modes
# ============================================================================
def kappa_free_free(ne, te, freq):
    """Thermal free-free absorption coefficient [cm^-1] (Dulk 1985, Eq. 21):
    kappa = 9.78e-3 n_e^2 / (nu^2 T^1.5) * G, G = 24.5 + ln T - ln nu (T > 2e5 K)."""
    G = np.where(te < 2e5, 18.2 + np.log(te**1.5) - np.log(freq),
                 24.5 + np.log(te) - np.log(freq))
    return 9.78e-3 * ne**2 / (freq**2 * te**1.5) * np.maximum(G, 1.0)


def kappa_two_modes(ne, te, Ymi, cos_theta, freq):
    """In a field the x-mode is absorbed more, the o-mode less:
    kappa_x = kappa/(1 - q)^2, kappa_o = kappa/(1 + q)^2, q = Y |cos(theta)| (<= 0.9)."""
    kappa = kappa_free_free(ne, te, freq)
    q = np.minimum(Ymi * np.abs(cos_theta), 0.9)
    return kappa / (1.0 - q)**2, kappa / (1.0 + q)**2


# ============================================================================
# STEP 5. Gyroresonance layers and their optical depth
# ============================================================================
def axial_ratio_x(Xmi, Ymi, cos_theta):
    """|T_x|: how circular (1) or linear (0) the x-mode is."""
    c = np.abs(cos_theta)
    s2 = 1.0 - c * c
    a = np.maximum(1.0 - Xmi, 1e-12)
    den = Ymi * s2 + np.sqrt(Ymi**2 * s2**2 + 4.0 * a**2 * c**2)
    return np.where(den > 0, 2.0 * a * c / np.where(den > 0, den, 1.0), 1.0)


def mode_factor(Xmi, s, cos_theta, mode):
    """F_x = (1 + |T_x| |cos|)^2/(1 + T_x^2), F_o = (|T_x| - |cos|)^2/(1 + T_x^2),
    with T_x evaluated on the layer, where Y = 1/s."""
    c = np.abs(cos_theta)
    tx = axial_ratio_x(Xmi, 1.0 / s, c)
    if mode == "x":
        return (1.0 + tx * c)**2 / (1.0 + tx**2)
    return (tx - c)**2 / (1.0 + tx**2)


def layer_tau(s, ne, te, cos_theta, l_b, freq, Xmi, mode):
    """Optical depth of the s-th harmonic layer (cgs):
    tau = (pi e^2/m_e c) (n_e L_B/nu) (s^2/s!) (s^2 beta^2 sin^2/2)^(s-1) F."""
    beta2 = KB * te / (ME * C**2)
    sin2 = np.maximum(1.0 - cos_theta**2, 0.0)
    F = mode_factor(Xmi, s, cos_theta, mode)
    return (GR_PREFACTOR * ne * l_b / freq * s**2 / math.factorial(s)
            * (0.5 * s**2 * beta2 * sin2)**(s - 1) * F)


def layers_in_steps(ne, te, b, cos_theta, nu_b, Xmi, dl, freq, s_max=6):
    """For every step k (between nodes k and k+1) and every harmonic s, find out
    whether the layer |B| = nu/(s nu_B per gauss) lies inside the step, and add
    its optical depth. Returns, per step and per mode, the layer optical depth
    and (tau * T) for the source function."""
```

```
    shape = dl.shape
    tau_x = np.zeros(shape); tau_o = np.zeros(shape)
    tT_x = np.zeros(shape); tT_o = np.zeros(shape)
    f0, f1 = nu_b[..., :-1], nu_b[..., 1:] # nu_B at both ends
    for s in range(1, s_max + 1):
        crosses = (s * f0 - freq) * (s * f1 - freq) < 0 # sign change -> a layer
        if not crosses.any():
            continue
        u = (freq / s - f0) / np.where(crosses, f1 - f0, 1.0) # where inside the step
        u = np.where(crosses, u, 0.0)
        at = lambda a: a[..., :-1] + u * (a[..., 1:] - a[..., :-1]) # value at the layer
        ne_c, te_c, cos_c, X_c = at(ne), at(te), at(cos_theta), at(Xmi)
        b_s = freq / (s * NU_B_PER_GAUSS) # |B| on the layer
        l_b = b_s * dl / np.maximum(np.abs(b[..., 1:] - b[..., :-1]), 1e-30) # L_B [cm]
        tx = np.where(crosses, layer_tau(s, ne_c, te_c, cos_c, l_b, freq, X_c, "x"), 0.0)
        to = np.where(crosses, layer_tau(s, ne_c, te_c, cos_c, l_b, freq, X_c, "o"), 0.0)
        if s == 1: # the x-mode cannot reach s = 1
            tx = np.where(X_c > 0, 0.0, tx)
        tau_x += tx; tau_o += to
        tT_x += tx * te_c; tT_o += to * te_c
    return tau_x, tau_o, tT_x, tT_o


# ============================================================================
# STEP 6. Radiative transfer for right- and left-hand circular polarisation
# ============================================================================
def transfer(dtau, tT, cutoff):
    """Walk along the line from the observer inwards and add the light of every
    step, dimmed by everything in front of it:

        T_p += T_step (1 - exp(-dtau)) exp(-tau), tau += dtau

    T_step = sum(tau_i T_i)/sum(tau_i) is the temperature the step glows at.
    Works in temperature units (T_p = I_p c^2/(k nu^2), one polarisation).
    A polarisation stops at its first cutoff node."""
    npx, npy, nk = dtau.shape
    t_pol = np.zeros((npx, npy))
    tau = np.zeros((npx, npy))
    alive = ~cutoff[..., 0]
    for k in range(nk): # step k: node k -> k+1
        alive &= ~cutoff[..., k + 1]
        d = np.where(alive, dtau[..., k], 0.0)
        t_step = np.where(d > 0, tT[..., k] / np.where(d > 0, d, 1.0), 0.0)
        t_pol += t_step * (-np.expm1(-d)) * np.exp(-tau)
        tau += d
    return t_pol, tau


def gyroresonance_image(freq, s_max=6):
    """Everything together for one frequency [Hz]. Returns a dict of maps."""
    c, X, Y, z = lines_of_sight()
    ne, te, b, cos_theta, nu_b, Xmi, Ymi = sample(X, Y, z, freq)
    dl = np.abs(np.diff(z)) * MM * np.ones_like(ne[..., 1:]) # step lengths [cm]

    # mid-step values for the free-free part
    mid = lambda a: 0.5 * (a[..., 1:] + a[..., :-1])
    kx, ko = kappa_two_modes(mid(ne), mid(te), mid(Ymi), mid(cos_theta), freq)
    dtau_x, dtau_o = kx * dl, ko * dl
    tT_x, tT_o = dtau_x * mid(te), dtau_o * mid(te)

    # the layers
    gx, go, gTx, gTo = layers_in_steps(ne, te, b, cos_theta, nu_b, Xmi, dl, freq, s_max)
    dtau_x, dtau_o = dtau_x + gx, dtau_o + go
```

```
    tT_x, tT_o = tT_x + gTx, tT_o + gTo

    # right-hand (R) light is the x-mode where B points towards us (cos > 0), else the o-mode
    pos = mid(cos_theta) > 0
    dtau_R, tT_R = np.where(pos, dtau_x, dtau_o), np.where(pos, tT_x, tT_o)
    dtau_L, tT_L = np.where(pos, dtau_o, dtau_x), np.where(pos, tT_o, tT_x)
    cut_x, cut_o = Xmi >= 1.0 - Ymi, Xmi >= 1.0 # where the modes cannot exist
    cut_R = np.where(cos_theta > 0, cut_x, cut_o)
    cut_L = np.where(cos_theta > 0, cut_o, cut_x)
    t_R, tau_R = transfer(dtau_R, tT_R, cut_R)
    t_L, tau_L = transfer(dtau_L, tT_L, cut_L)

    # ============================================================================
    # STEP 7. What the telescope sees
    # ============================================================================
    tb = 0.5 * (t_R + t_L) # T_b of Stokes I
    tb_v = 0.5 * (t_R - t_L) # T_b of Stokes V
    pol = np.where(tb > 0, tb_v / np.where(tb > 0, tb, 1.0), 0.0)
    # flux density at the Earth: S = (2 k nu^2/c^2) sum(T_b) dOmega
    d_omega = (FOV / NPIX * MM / 1.496e13)**2 # pixel solid angle [sr]
    flux = 2.0 * KB * freq**2 / C**2 * tb.sum() * d_omega / 1e-19 # in sfu
    flux_v = 2.0 * KB * freq**2 / C**2 * tb_v.sum() * d_omega / 1e-19
    return dict(x=c, tb=tb, tb_v=tb_v, pol=pol, tb_R=t_R, tb_L=t_L,
                tau_R=tau_R, tau_L=tau_L, flux_sfu=flux, flux_v_sfu=flux_v)


# ============================================================================
if __name__ == "__main__":
    freqs = [5e9, 17e9]
    res = {f: gyroresonance_image(f) for f in freqs}
    for f, r in res.items():
        print(f"{f / 1e9:5.1f} GHz: max T_b = {r['tb'].max() / 1e6:5.2f} MK, "
              f"flux = {r['flux_sfu']:7.2f} sfu, V flux = {r['flux_v_sfu']:+7.2f} sfu")

    ext = [-0.5 * FOV, 0.5 * FOV, -0.5 * FOV, 0.5 * FOV]
    fig, ax = plt.subplots(2, 2, figsize=(9.2, 7.6))
    panels = [(res[5e9]["tb"] / 1e6, "5 GHz: $T_b$ [MK]", "inferno", 0, None),
              (res[5e9]["pol"], "5 GHz: $V/I$", "RdBu_r", -1, 1),
              (res[17e9]["tb"] / 1e6, "17 GHz: $T_b$ [MK]", "inferno", 0, None),
              (res[17e9]["pol"], "17 GHz: $V/I$", "RdBu_r", -1, 1)]
    for a, (data, title, cmap, lo, hi) in zip(ax.ravel(), panels):
        im = a.imshow(data, origin="lower", extent=ext, cmap=cmap, vmin=lo, vmax=hi)
        a.set_title(title, fontsize=10)
        a.set_xlabel("x [Mm]")
        a.set_ylabel("y [Mm]")
        fig.colorbar(im, ax=a, shrink=0.85)
    fig.tight_layout()
    fig.savefig("gyroresonance_simple_fig.png", dpi=130)
    fig.savefig("gyroresonance_simple_fig.pdf")
    print("written: gyroresonance_simple_fig.png / .pdf")
```